

Relatório de Vulnerabilidades

Sistema Integrado de Atendimento e Execução de Serviços para Oficina Mecânica

Grupo	Grupo 97
Participante	Pedro Paulo dos Santos Silva
Username no Discord	pedropaulo6447
Repositório privado	https://github.com/ppdssdev/tech-challenge-oficina
Data do scan	30/06/2026
Ferramenta principal	Trivy via Docker

Este relatório documenta a análise de vulnerabilidades realizada no projeto, as vulnerabilidades encontradas no scan inicial, as correções aplicadas e o resultado final após mitigação.

Tech Challenge FIAP

1. Objetivo

Este documento apresenta o relatório de vulnerabilidades do MVP backend da oficina mecânica. A análise teve como foco as dependências declaradas no projeto, a imagem Docker final da API, os pacotes do sistema operacional da imagem e as dependências Java empacotadas no JAR da aplicação.

O objetivo foi identificar vulnerabilidades conhecidas, reduzir a superfície de ataque da imagem final e registrar de forma transparente qualquer risco residual ainda presente no momento da validação.

2. Ferramenta Utilizada

Foi utilizado o Trivy via Docker para executar os scans de filesystem e de imagem Docker. A execução via Docker evita a necessidade de instalação local da ferramenta e facilita a reprodutibilidade do processo.

- Scan das dependências declaradas no projeto.
- Scan da imagem Docker final da API.
- Scan dos pacotes do sistema operacional da imagem.
- Scan das dependências Java empacotadas no JAR da aplicação.

3. Comandos Executados

3.1 Scan do filesystem

```
docker run --rm -v "$PWD:/workspace" -v "$HOME/.cache/trivy:/root/.cache" \
  aquasec/trivy:latest fs --offline-scan --format table \
  --output /workspace/target/security/trivy-fs.txt /workspace
```

3.2 Build da imagem Docker

```
docker build --progress=plain -t oficina-api:local .
```

3.3 Scan da imagem Docker

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v "$HOME/.cache/trivy:/root/.cache" \
  -v "$PWD/target/security:/reports" \
  aquasec/trivy:latest image --offline-scan --format table \
  --output /reports/trivy-image.txt oficina-api:local
```

4. Resultado Inicial

O primeiro scan da imagem encontrou vulnerabilidades relevantes em dependências da aplicação e na imagem base utilizada inicialmente. O resultado inicial apontou uma superfície de ataque maior, principalmente por conta da imagem base e de versões antigas de dependências do ecossistema Spring, Tomcat, Jackson e PostgreSQL.

Alvo	Resultado inicial
Imagem Ubuntu base	106 vulnerabilidades, sem críticas ou altas
JAR da aplicação	43 vulnerabilidades, incluindo altas e críticas
Binário auxiliar da imagem	13 vulnerabilidades

4.1 Principais Pontos Encontrados

- tomcat-embed-core com vulnerabilidades críticas e altas.
- spring-security-web, spring-core, spring-webmvc e spring-boot com vulnerabilidades reportadas.

- Driver PostgreSQL com vulnerabilidades altas.
- Imagem baseada em Ubuntu com maior superfície de pacotes instalados.

5. Correções Aplicadas

Após o resultado inicial, foram aplicadas atualizações de dependências, ajustes na imagem base e medidas de hardening no Dockerfile.

Ação	Motivo
Atualização do Spring Boot de 3.5.0 para 3.5.14	Receber versões corretivas do ecossistema Spring, Hibernate, Tomcat e plugins gerenciados pelo BOM.
Override de tomcat.version para 10.1.55	Remover vulnerabilidades críticas e altas indicadas no Tomcat.
Override de postgresql.version para 42.7.11	Atualizar o driver JDBC para versão corrigida.
Override de commons-lang3.version para 3.18.0	Remover vulnerabilidade reportada em Commons Lang 3.17.0.
Override de jackson-bom.version para 2.21.4	Remover vulnerabilidades altas reportadas em Jackson 2.21.2.
Troca da imagem final para eclipse-temurin:21-jre-alpine	Reduzir a superfície de ataque da imagem final.
Execução de apk upgrade --no-cache no build da imagem	Atualizar pacotes Alpine com versão corrigida disponível.
Execução da aplicação como usuário não-root	Reduzir impacto em caso de comprometimento do processo.

6. Resultado Final

6.1 Scan do filesystem

Resultado final do scan em pom.xml:

pom.xml - 0 vulnerabilidades detectadas

6.2 Scan da imagem Docker

Resultado final da imagem oficina-api:local:

Alvo	Vulnerabilidades finais
Sistema operacional Alpine 3.23.5	0
JAR da aplicação	1

6.3 Severidade final do JAR

Severidade	Quantidade
Crítica	0
Alta	0
Média	1
Baixa	0

6.4 Vulnerabilidade residual

Biblioteca	CVE	Severidade	Versão instalada	Versão corrigida indicada
com.fasterxml.jackson.core:jackson-databind	CVE-2026-54515	Média	2.21.4	3.1.4, 2.18.9, 2.21.5

7. Análise do Risco Residual

A única vulnerabilidade residual ficou em jackson-databind, com severidade média. Segundo o resultado do Trivy, uma das versões corrigidas indicadas seria a 2.21.5. No momento da validação, essa versão do jackson-bom não estava disponível no Maven Central para atualização direta do projeto. A alternativa 2.18.9 também foi testada, mas o artifact jackson-bom:2.18.9 não estava disponível.

A decisão tomada foi manter jackson-bom.version=2.21.4, pois essa versão removeu vulnerabilidades altas anteriormente reportadas e deixou o projeto sem vulnerabilidades críticas ou altas no resultado final.

Decisão registrada:

- Manter jackson-bom.version=2.21.4 no momento da entrega.
- Registrar a vulnerabilidade residual no relatório.
- Reexecutar o scan antes da entrega final sempre que houver nova versão disponível.
- Atualizar para 2.21.5 ou versão corrigida compatível assim que estiver disponível no Maven Central.

8. Controles de Segurança Implementados

Além da correção de dependências e da redução da superfície da imagem Docker, o sistema possui controles de segurança implementados na própria aplicação.

Controle	Descrição
JWT nas APIs administrativas	Rotas administrativas exigem autenticação por token JWT.
Endpoints públicos restritos	A API pública é limitada ao acompanhamento da OS por código e documento do cliente.
Tratamento padronizado de erros	Erros são convertidos para status HTTP adequados, como 400, 401, 403, 404, 409 e 422.
Validação de CPF/CNPJ	O Value Object DocumentNumber valida e normaliza o documento no domínio.
Validação de placa	O Value Object VehiclePlate valida e normaliza placas brasileiras.
Controle de concorrência no estoque	Uso de @Version e lock pessimista em operações críticas de estoque.
Usuário não-root no Dockerfile	A aplicação não executa como root na imagem final.
Schema versionado	O banco PostgreSQL usa Flyway para controle de migrations.

9. Recomendações Futuras

- Reexecutar o Trivy antes da entrega final e sempre que houver alteração relevante no pom.xml ou no Dockerfile.
- Atualizar jackson-bom.version para 2.21.5 ou versão corrigida compatível assim que estiver disponível.
- Externalizar segredos de produção via secret manager ou variáveis de ambiente seguras.

- Substituir a senha padrão do admin em qualquer ambiente que não seja local.
- Adicionar pipeline CI com mvn verify e scan Trivy automático.
- Bloquear merge quando forem identificadas vulnerabilidades críticas ou altas.
- Armazenar os relatórios de scan em pasta versionada de documentação ou anexá-los à entrega.

9.1 Comando recomendado para novo scan da imagem

```
docker build --progress=plain -t oficina-api:local .
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  -v "$HOME/.cache/trivy:/root/.cache" \
  aquasec/trivy:latest image oficina-api:local
```

10. Organização Recomendada dos Arquivos no Repositório

Para a entrega, recomenda-se manter este PDF dentro da pasta docs e, caso os arquivos de saída do Trivy sejam versionados, mantê-los em uma subpasta de segurança.

```
docs/
├── relatorio-vulnerabilidades.pdf
└── security/
    ├── trivy-fs.txt
    └── trivy-image.txt
```

Caso a pasta target esteja no .gitignore, os arquivos trivy-fs.txt e trivy-image.txt podem ser copiados de target/security para docs/security antes do commit.

11. Conclusão

O scan inicial identificou vulnerabilidades relevantes em dependências da aplicação e na imagem base. Após as correções, o resultado final ficou sem vulnerabilidades críticas ou altas, com 0 vulnerabilidades no pom.xml, 0 vulnerabilidades no sistema operacional Alpine da imagem final e apenas 1 vulnerabilidade média residual no JAR da aplicação.

A vulnerabilidade residual foi documentada com justificativa técnica e recomendação de atualização futura. Dessa forma, o projeto apresenta uma postura de segurança adequada para um MVP acadêmico, com dependências atualizadas, imagem final reduzida, aplicação executando como usuário não-root, APIs administrativas protegidas por JWT e validações de domínio para dados sensíveis como documento do cliente e placa do veículo.